

A research and trading platform

A Unified Quantitative Research Pipeline

Five components, one architecture

Pattern memory · RMT correlation cleaning · TDA regime detection
Ising portfolio optimization · AI agent reasoning

Technical design document with mathematical foundations

Contents

1	Introduction	3
1.1	Pipeline overview	3
1.2	Document structure	4
2	Stage 1: Pattern Memory Engine	4
2.1	What it does	4
2.2	Mathematical foundations	4
2.2.1	Graph schema	5
2.3	Implementation	5
2.3.1	LLM extraction	5
2.3.2	Pattern detection job	6
2.3.3	Technology stack	6
2.4	Integration with downstream stages	6
3	Stage 2: Random Matrix Theory Correlation Cleaning	6
3.1	What it does	6
3.2	Mathematical foundations	7
3.2.1	The Marchenko-Pastur distribution	7
3.2.2	Cleaning procedure	7
3.2.3	Diagonal preservation	8
3.3	Algorithm	8
3.4	Implementation	8
3.5	Integration with downstream stages	10
4	Stage 3: Topological Regime Detection	10
4.1	What it does	10
4.2	Mathematical foundations	10
4.2.1	Simplicial complexes from correlation matrices	10
4.2.2	Homology and persistence	10
4.2.3	Feature vectors from persistence diagrams	11
4.2.4	Regime classification	11
4.3	Pipeline	11
4.4	Algorithm	12
4.5	Implementation	12
4.6	Latency analysis	13
4.7	Integration with downstream stages	13
5	Stage 4: VOLTA — Ising Portfolio Optimization	13
5.1	What it does	13
5.2	Mathematical foundations	13
5.2.1	Binary variables and the QUBO formulation	13
5.2.2	Standard constraint penalties	13
5.2.3	Regime-conditional edges	14
5.2.4	Strategy-type diversity	14
5.3	Full objective	14
5.4	Equivalence to Ising model	14
5.5	Solver: simulated annealing	15
5.6	Implementation	15
5.7	Solver comparison	16

5.8	Integration with downstream stages	16
6	Stage 5: AI Agent Meta Layer	17
6.1	What it does	17
6.2	Architecture	17
6.3	Validation logic	17
6.4	Implementation	17
7	Cross-Cutting Concerns	17
7.1	Data architecture	17
7.2	Repository structure	18
7.3	Reproducibility	19
7.4	Testing strategy	19
8	Build Sequence	19
9	Validation Methodology	19
9.1	Stage-by-stage validation	19
9.2	End-to-end validation	20
	Summary	20

1 Introduction

This document specifies a unified quantitative research pipeline combining five distinct techniques into one integrated system. The five components are: a pattern memory engine that accumulates observations over time, a random matrix theory filter for correlation cleaning, a topological data analysis module for regime detection, an Ising-based portfolio optimizer named VOLTA, and a cross-cutting AI agent layer providing validation and synthesis.

The pipeline is designed to be market-agnostic. While many examples reference Indian equity and derivatives markets, the architecture supports any market with sufficient data quality. Adapters at the data layer normalize heterogeneous inputs into a common internal format that downstream stages consume.

Design principle

Each component is independently valuable, but they form a pipeline where outputs of earlier stages improve the quality of later stages. The pattern memory provides context for AI agents. RMT cleaning produces denoised correlation matrices that TDA uses. TDA regime detection conditions VOLTA's optimization objectives. AI agents validate outputs at every stage.

1.1 Pipeline overview

Figure 1 shows the high-level architecture. Data flows from heterogeneous sources through the pattern memory engine, which accumulates and stores observations. Time-series quantitative data flows through RMT cleaning into TDA regime detection. The regime signal conditions the VOLTA optimizer, which produces trade decisions. AI agents observe and reason about outputs at every stage.

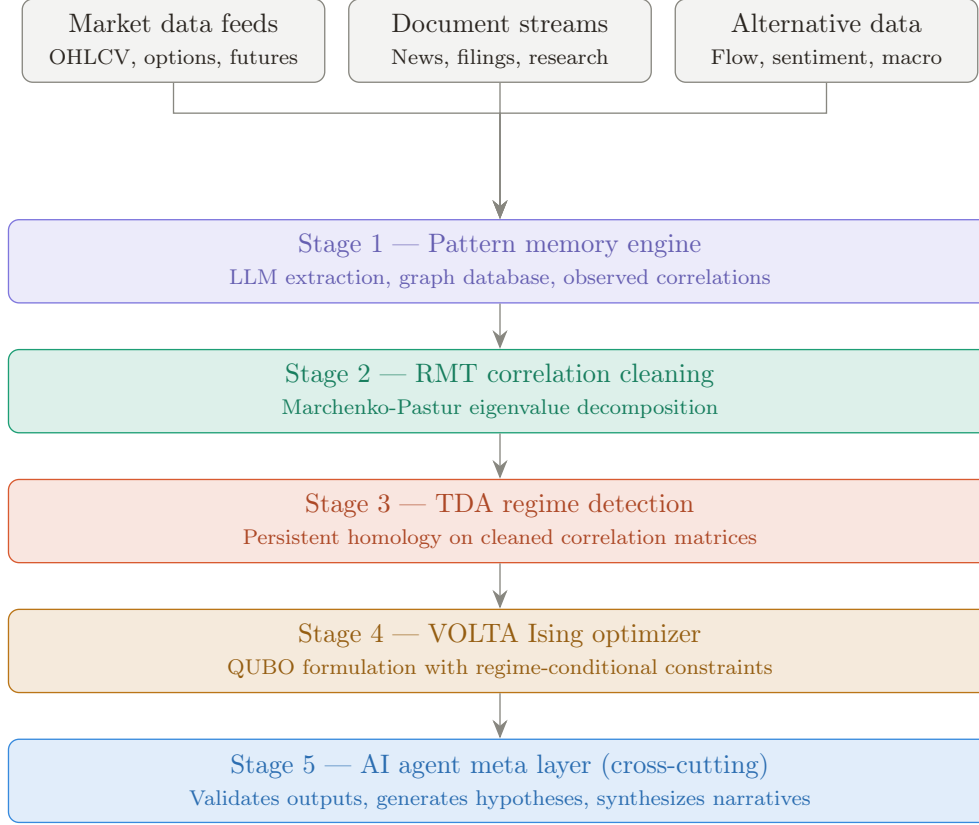


Figure 1: The unified pipeline. Each stage consumes outputs of previous stages and produces outputs for subsequent stages. The AI agent layer at the bottom observes all stages.

1.2 Document structure

Sections 2 through 6 cover the five stages in detail, each with mathematical foundations, implementation strategy, and integration points. Section 7 covers cross-cutting concerns including data architecture, testing strategy, and reproducibility. Section 8 provides a 12-week build sequence. Section 9 discusses validation methodology.

2 Stage 1: Pattern Memory Engine

2.1 What it does

The pattern memory engine accumulates structured observations from heterogeneous unstructured sources. It extracts entities (companies, sectors, people, events), relationships (mentioned-in, occurred-on, preceded-by, correlated-with), and metadata (timestamps, sources, confidence) into a queryable graph database. Over months and years of operation, accumulated observations enable pattern detection that no single document or session could reveal.

Core idea

This stage is not predictive. It is associative. It learns that “when X happens, Y often follows” without claiming to understand causation. The value compounds over time because every new observation is connected to all previous ones in the graph.

2.2 Mathematical foundations

The data model is a heterogeneous graph $G = (V, E)$ where:

- $V = V_E \cup V_T \cup V_C$ is the set of nodes, partitioned into entities V_E , temporal events V_T , and conceptual themes V_C .
- $E \subseteq V \times V \times \mathcal{R} \times \mathbb{R}^d$ is the set of edges, where each edge carries a relationship type from $\mathcal{R}$ and a feature vector in $\mathbb{R}^d$ containing timestamp, confidence, and source identifiers.

Pattern detection operates on G by computing graph statistics. The simplest pattern is co-occurrence frequency. For entities $a, b \in V_E$ and a time window $[t_1, t_2]$:

$$\text{cooccur}(a, b, t_1, t_2) = |\{(e_1, e_2) \in E^2 : e_1 = (a, v, r_1, \cdot), e_2 = (b, v, r_2, \cdot), t(v) \in [t_1, t_2]\}| \quad (1)$$

where v is a shared event or document node and $t(v)$ is the timestamp of v .

More interesting is the temporal lead-lag pattern. For entities a and b , define the temporal correlation at lag τ :

$$\rho_{a,b}(\tau) = \frac{\sum_t (n_a(t) - \bar{n}_a)(n_b(t + \tau) - \bar{n}_b)}{\sqrt{\sum_t (n_a(t) - \bar{n}_a)^2 \sum_t (n_b(t) - \bar{n}_b)^2}} \quad (2)$$

where $n_a(t)$ is the count of mentions of entity a at time t . If $\rho_{a,b}(\tau)$ is significantly positive for $\tau > 0$, then mentions of a tend to precede mentions of b by τ time units.

2.2.1 Graph schema

Figure 2 shows the core schema. Companies, sectors, events, themes, and management teams are first-class nodes. Documents are also nodes, but they primarily serve as evidence carriers — every relationship edge points to the document that justified its creation.

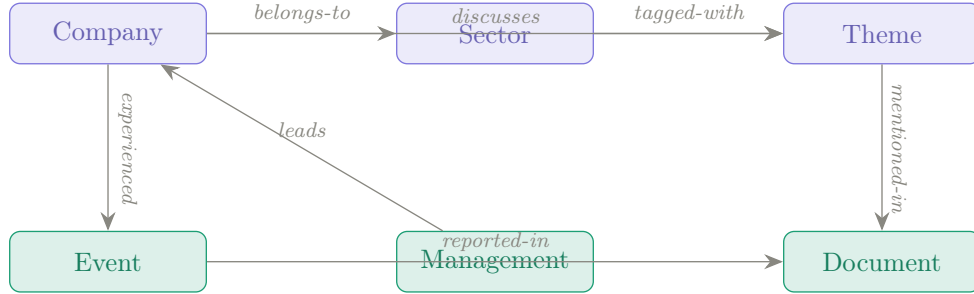


Figure 2: Graph schema for the pattern memory engine. Nodes are entities or events; edges carry relationship type, timestamp, source document reference, and confidence score.

2.3 Implementation

2.3.1 LLM extraction

Documents arrive as unstructured text. An LLM converts them into structured observations. The extraction prompt is conservative — only extract claims that are literally stated in the source, attach verbatim quotes as evidence, and assign confidence scores.

For each document d , the extractor produces a tuple set:

$$T_d = \{(s, p, o, q, c) : s, o \in V, p \in \mathcal{R}, q \in \text{quote}(d), c \in [0, 1]\} \quad (3)$$

where each tuple represents a subject-predicate-object triple, the supporting quote q from the document, and a confidence c .

2.3.2 Pattern detection job

A scheduled job runs over the graph at fixed intervals (typically weekly), computing patterns of interest. Algorithm 1 shows the core logic.

Algorithm 1 Periodic pattern detection over the memory graph

Require: Graph $G = (V, E)$, time window W , significance threshold α

Ensure: Set of discovered patterns P

```

1:  $P \leftarrow \emptyset$ 
2: for each entity pair  $(a, b) \in V_E \times V_E$  with  $a \neq b$  do
3:    $\rho \leftarrow$  compute temporal correlation (Eq. 2) over window  $W$ 
4:   if  $|\rho| > \rho_{\min}$  and  $p\text{-value}(\rho) < \alpha$  then
5:      $P \leftarrow P \cup \{(a, b, \rho, \text{evidence})\}$ 
6:   end if
7: end for
8: for each theme  $t \in V_C$  do
9:    $f \leftarrow$  mention frequency of  $t$  in window  $W$ 
10:   $\bar{f} \leftarrow$  historical mean mention frequency
11:  if  $f > \bar{f} + 2\sigma$  then
12:     $P \leftarrow P \cup \{(t, \text{trending}, f/\bar{f})\}$ 
13:  end if
14: end for
15: return  $P$ 

```

2.3.3 Technology stack

- **Graph database:** Neo4j Community Edition. Supports Cypher query language, scales to millions of nodes on a single machine.
- **Relational complement:** PostgreSQL with TimescaleDB extension for raw documents and time-series companion data.
- **LLM client:** Anthropic Claude API (preferred for structured extraction) or OpenAI API.
- **Orchestration:** Python with FastAPI for the service interface, Celery or simple cron for scheduled jobs.
- **Language:** Python 3.11+ throughout.

2.4 Integration with downstream stages

The pattern memory engine serves as context provider. When VOLTA evaluates candidate trades, it queries the memory for recent events involving the relevant entities. When the TDA layer flags a regime change, the agent layer queries the memory for documented context that might explain it. The graph is read-heavy for downstream stages; only the extraction job writes.

3 Stage 2: Random Matrix Theory Correlation Cleaning

3.1 What it does

Standard sample correlation matrices computed from finite time series contain substantial noise. Random matrix theory tells us exactly how much. For an $N \times T$ return matrix where T is not vastly larger than N , the eigenvalues of the sample correlation matrix follow the Marchenko-Pastur distribution in their noisy regime. Eigenvalues outside this bulk distribution contain

genuine signal. RMT cleaning removes the noise eigenvalues, producing a denoised correlation matrix that downstream stages use.

Why this matters

A retail trader using 60 days of data on 50 stocks has $T/N = 1.2$. Their sample correlation matrix is approximately 70–80% noise. Portfolio construction decisions based on such matrices are statistically meaningless. RMT cleaning is one of the cheapest, most reliable improvements available.

3.2 Mathematical foundations

3.2.1 The Marchenko-Pastur distribution

Theorem 3.1 (Marchenko-Pastur). Let $\mathbf{X}$ be an $N \times T$ matrix with i.i.d. entries of mean 0 and variance σ^2 . As $N, T \rightarrow \infty$ with $N/T \rightarrow q$ for some $q \in (0, 1)$, the empirical spectral distribution of the sample covariance matrix $\mathbf{S} = \frac{1}{T} \mathbf{X} \mathbf{X}^\top$ converges to the Marchenko-Pastur distribution with density:

$$\rho_{MP}(\lambda) = \frac{1}{2\pi\sigma^2 q \lambda} \sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)} \quad (4)$$

on the support $[\lambda_-, \lambda_+]$, where:

$$\lambda_{\pm} = \sigma^2(1 \pm \sqrt{q})^2 \quad (5)$$

Figure 3 shows the Marchenko-Pastur density for $q = 0.5$ and $\sigma^2 = 1$.

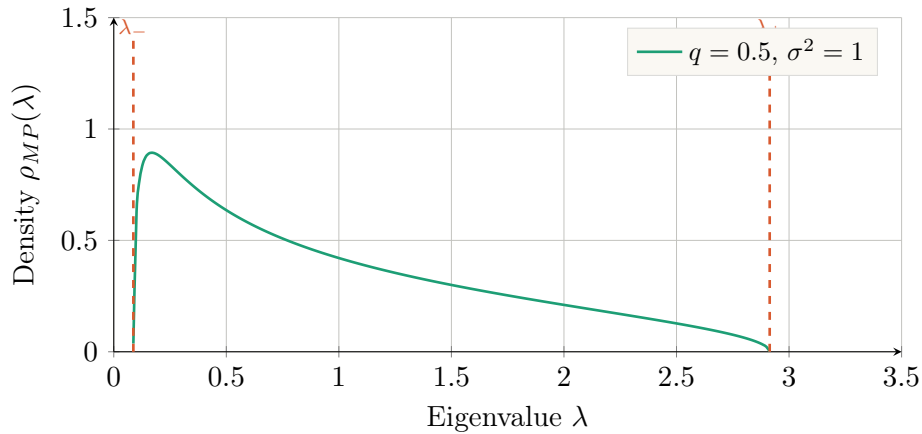


Figure 3: The Marchenko-Pastur density for $q = N/T = 0.5$ and noise variance $\sigma^2 = 1$. Eigenvalues falling within $[\lambda_-, \lambda_+]$ are statistically indistinguishable from pure noise. Eigenvalues above λ_+ carry signal.

3.2.2 Cleaning procedure

Let $\mathbf{R}$ be an $N \times T$ matrix of standardized returns. Compute the sample correlation matrix:

$$\mathbf{C} = \frac{1}{T} \mathbf{R} \mathbf{R}^\top \quad (6)$$

Perform eigendecomposition:

$$\mathbf{C} = \sum_{i=1}^N \lambda_i \mathbf{v}_i \mathbf{v}_i^\top \quad (7)$$

where $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$ are eigenvalues with corresponding eigenvectors $\mathbf{v}_i$.

Identify signal eigenvalues using the Marchenko-Pastur upper bound. With noise variance approximated by the bulk of the spectrum (excluding the largest market-mode eigenvalue), define:

$$\sigma_{\text{noise}}^2 = \frac{1}{N - k_{\text{signal}}} \sum_{i: \lambda_i \in [\lambda_-, \lambda_+]} \lambda_i \quad (8)$$

Replace each noise eigenvalue with the average of the noise bulk, preserving the trace:

$$\tilde{\lambda}_i = \begin{cases} \lambda_i & \text{if } \lambda_i > \lambda_+ \\ \bar{\lambda}_{\text{noise}} & \text{otherwise} \end{cases} \quad (9)$$

The cleaned correlation matrix is reconstructed as:

$$\tilde{\mathbf{C}} = \sum_{i=1}^N \tilde{\lambda}_i \mathbf{v}_i \mathbf{v}_i^\top \quad (10)$$

3.2.3 Diagonal preservation

The reconstruction above can produce off-diagonal entries with values that, while improved, leave diagonal entries deviated from unity. A final normalization step rescales:

$$\tilde{C}_{ij}^{\text{final}} = \frac{\tilde{C}_{ij}}{\sqrt{\tilde{C}_{ii} \tilde{C}_{jj}}} \quad (11)$$

This ensures $\tilde{C}_{ii}^{\text{final}} = 1$ for all i , maintaining the structural properties of a correlation matrix.

3.3 Algorithm

3.4 Implementation

The implementation is approximately 150 lines of Python. Listing 1 shows the core function.

```

1 import numpy as np
2 from numpy.linalg import eig
3
4 def rmt_clean(returns: np.ndarray) -> np.ndarray:
5     """
6     Clean a return matrix using Marchenko-Pastur eigenvalue filtering.
7
8     Args:
9         returns: (N, T) matrix of returns. N assets, T observations.
10
11     Returns:
12         (N, N) cleaned correlation matrix.
13     """
14     N, T = returns.shape
15     q = N / T
16
17     # Standardize each row
18     R = (returns - returns.mean(axis=1, keepdims=True))
19     R = R / R.std(axis=1, keepdims=True)
20

```

Algorithm 2 RMT-based correlation cleaning**Require:** Return matrix $\mathbf{R} \in \mathbb{R}^{N \times T}$ **Ensure:** Cleaned correlation matrix $\tilde{\mathbf{C}} \in \mathbb{R}^{N \times N}$

- 1: Standardize $\mathbf{R}$ so each row has zero mean and unit variance
- 2: Compute sample correlation $\mathbf{C} = \frac{1}{T} \mathbf{R} \mathbf{R}^\top$
- 3: Eigendecompose $\mathbf{C} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^\top$
- 4: Set $q \leftarrow N/T$
- 5: Compute Marchenko-Pastur bounds $\lambda_{\pm} = (1 \pm \sqrt{q})^2$
- 6: Identify signal eigenvalues: $S \leftarrow \{i : \lambda_i > \lambda_+\}$
- 7: Compute mean noise eigenvalue: $\bar{\lambda} \leftarrow \text{mean}(\{\lambda_i : i \notin S\})$
- 8: **for** $i = 1$ to N **do**
- 9: **if** $i \in S$ **then**
- 10: $\tilde{\lambda}_i \leftarrow \lambda_i$
- 11: **else**
- 12: $\tilde{\lambda}_i \leftarrow \bar{\lambda}$
- 13: **end if**
- 14: **end for**
- 15: Reconstruct $\tilde{\mathbf{C}} \leftarrow \mathbf{V} \tilde{\mathbf{\Lambda}} \mathbf{V}^\top$
- 16: Normalize: $\tilde{C}_{ij} \leftarrow \tilde{C}_{ij} / \sqrt{\tilde{C}_{ii} \tilde{C}_{jj}}$
- 17: **return** $\tilde{\mathbf{C}}$

```

21  # Sample correlation matrix
22  C = (R @ R.T) / T
23
24  # Eigendecomposition (ascending order from eigh)
25  eigenvalues, eigenvectors = eigh(C)
26
27  # Marchenko-Pastur bounds (assuming unit noise variance)
28  lambda_plus = (1 + np.sqrt(q))**2
29  lambda_minus = (1 - np.sqrt(q))**2
30
31  # Identify signal eigenvalues
32  signal_mask = eigenvalues > lambda_plus
33  n_signal = signal_mask.sum()
34
35  if n_signal == 0:
36      return np.eye(N) # nothing but noise
37
38  # Replace noise eigenvalues with their mean
39  noise_mean = eigenvalues[~signal_mask].mean()
40  cleaned_eigenvalues = np.where(signal_mask, eigenvalues, noise_mean)
41
42  # Reconstruct correlation matrix
43  C_clean = eigenvectors @ np.diag(cleaned_eigenvalues) @ eigenvectors.T
44
45  # Normalize to ensure unit diagonal
46  d = np.sqrt(np.diag(C_clean))
47  C_clean = C_clean / np.outer(d, d)
48
49  return C_clean

```

Listing 1: RMT correlation cleaning, core function

3.5 Integration with downstream stages

The cleaned correlation matrices feed directly into TDA regime detection (Section 4) and into VOLTA's risk calculations (Section 5). Rolling windows of cleaned matrices are stored as Parquet files indexed by timestamp.

4 Stage 3: Topological Regime Detection

4.1 What it does

Persistent homology characterizes the topological structure of data: how many connected components exist, how many loops, how many higher-dimensional voids, and at what scales these features appear and disappear. Applied to rolling correlation matrices, persistent homology produces features that detect structural regime changes in markets — often before classical methods based on volatility or returns.

The intuition

Imagine the correlation matrix as defining a distance between assets ($d = 1 - |\rho|$). Connect assets when their distance falls below a threshold ε . As ε grows from 0 to 1, the graph of connections evolves. Persistent homology tracks the topological features of this evolving graph. Different market regimes produce characteristically different topological signatures.

4.2 Mathematical foundations

4.2.1 Simplicial complexes from correlation matrices

Definition 4.1 (Vietoris-Rips complex). Given a finite point set X in a metric space and a scale parameter $\varepsilon \geq 0$, the Vietoris-Rips complex $\text{VR}_\varepsilon(X)$ is the abstract simplicial complex whose k -simplices are subsets of $k + 1$ points all pairwise within distance ε .

Given a cleaned correlation matrix $\tilde{C}$, define a distance:

$$d(i, j) = \sqrt{1 - \tilde{C}_{ij}^2} \quad (12)$$

This is a proper metric (positive, symmetric, satisfies triangle inequality) when $\tilde{C}$ is a valid correlation matrix. The Vietoris-Rips filtration $\{\text{VR}_\varepsilon\}_{\varepsilon \geq 0}$ defines a nested family of simplicial complexes growing as ε increases.

4.2.2 Homology and persistence

The k -th homology group $H_k(\text{VR}_\varepsilon)$ captures k -dimensional holes:

- H_0 counts connected components
- H_1 counts independent loops
- H_2 counts enclosed voids

The Betti number $\beta_k(\varepsilon) = \dim H_k(\text{VR}_\varepsilon)$ is the count of k -dimensional features at scale ε .

Definition 4.2 (Persistence diagram). The persistence diagram in dimension k is the multi-set:

$$\text{Dgm}_k = \{(b_i, d_i) : i \in I\} \quad (13)$$

where each pair represents a topological feature in dimension k that is born at scale b_i and dies at scale $d_i > b_i$. The persistence of a feature is $d_i - b_i$.

Features with high persistence are robust topological signals; features with low persistence are typically noise.

4.2.3 Feature vectors from persistence diagrams

Persistence diagrams are not vectors in $\mathbb{R}^d$, so we need a vectorization to feed into a classifier. Two standard approaches:

Betti curves discretize the Betti numbers as functions of scale:

$$B_k(\varepsilon) = \beta_k(\varepsilon) \quad (14)$$

The vector $[B_k(\varepsilon_1), B_k(\varepsilon_2), \dots, B_k(\varepsilon_m)]$ for a fixed grid of scales gives a fixed-dimension feature.

Persistence landscapes (Bubenik, 2015) define a sequence of functions $\lambda_k^{(\ell)} : \mathbb{R} \rightarrow \mathbb{R}$ from a persistence diagram:

$$\lambda_k^{(\ell)}(t) = \ell\text{-th largest value of } \{\Lambda_{(b_i, d_i)}(t) : (b_i, d_i) \in \text{Dgm}_k\} \quad (15)$$

where $\Lambda_{(b,d)}(t) = \max(0, \min(t - b, d - t))$ is a tent function over the interval $[b, d]$. The integrals of the landscapes serve as a stable, vectorized representation.

4.2.4 Regime classification

Given a sliding window of length W ending at time t , compute the cleaned correlation matrix $\tilde{\mathbf{C}}_t$ from returns in $[t - W, t]$. Build the Vietoris-Rips filtration, compute persistence diagrams in dimensions 0, 1, and 2, and vectorize to a feature $\mathbf{x}_t \in \mathbb{R}^D$.

A classifier $f : \mathbb{R}^D \rightarrow \{R_1, R_2, \dots, R_K\}$ maps feature vectors to one of K regime labels. The classifier is trained on historical periods with known regime labels (assigned by hindsight analysis).

For the classifier, gradient-boosted trees (XGBoost or LightGBM) or random forests typically outperform deep networks on these problem sizes. The feature vectors are dense, low-dimensional ($D \sim 100$), and training data is limited.

4.3 Pipeline

Figure 4 illustrates the TDA pipeline.

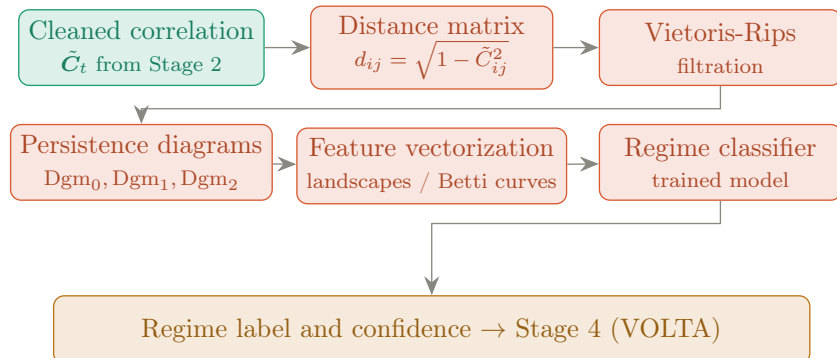


Figure 4: The TDA regime detection pipeline. Cleaned correlation matrices flow through topological feature extraction into a regime classifier whose output conditions VOLTA’s optimization.

Algorithm 3 TDA-based regime detection**Require:** Cleaned correlation matrix $\tilde{C}_t$, trained classifier f **Ensure:** Regime label $r_t \in \{R_1, \dots, R_K\}$ and confidence $c_t \in [0, 1]$

- 1: Compute distance matrix $d_{ij} \leftarrow \sqrt{1 - \tilde{C}_{ij}^2}$
- 2: Build Vietoris-Rips filtration up to maximum scale $\varepsilon_{\max}$
- 3: Compute persistence diagrams $\text{Dgm}_0, \text{Dgm}_1, \text{Dgm}_2$
- 4: Extract feature vector $\mathbf{x}_t$ via landscape integrals or Betti curves
- 5: $(r_t, c_t) \leftarrow f(\mathbf{x}_t)$
- 6: **return** (r_t, c_t)

4.4 Algorithm

4.5 Implementation

Library: `giotto-tda` is the primary library for production work. It builds on `gudhi` and provides scikit-learn-compatible transformers. Listing 2 shows the core extraction logic.

```

1  import numpy as np
2  from gttda.homology import VietorisRipsPersistence
3  from gttda.diagrams import PersistenceLandscape, Amplitude
4
5  def extract_tda_features(corr_matrix: np.ndarray) -> np.ndarray:
6      """
7      Compute persistent homology features from a correlation matrix.
8
9      Args:
10         corr_matrix: (N, N) cleaned correlation matrix.
11
12     Returns:
13         (D,) feature vector for downstream classification.
14     """
15     # Convert correlation to distance
16     distance_matrix = np.sqrt(1 - corr_matrix**2)
17
18     # Vietoris-Rips persistence in dimensions 0, 1, 2
19     vr = VietorisRipsPersistence(
20         metric="precomputed",
21         homology_dimensions=[0, 1, 2],
22         max_edge_length=1.0,
23     )
24     diagrams = vr.fit_transform(distance_matrix[None, :, :])
25
26     # Compute persistence landscapes
27     landscape = PersistenceLandscape(n_layers=5, n_bins=100)
28     landscape_features = landscape.fit_transform(diagrams)
29
30     # Compute amplitude metrics
31     amplitude = Amplitude(metric="wasserstein")
32     amplitude_features = amplitude.fit_transform(diagrams)
33
34     return np.concatenate([
35         landscape_features.flatten(),
36         amplitude_features.flatten(),
37     ])

```

Listing 2: TDA feature extraction using `giotto-tda`

4.6 Latency analysis

The headline research question for this stage is detection latency. For each historical regime change with known onset t_{true} :

$$\Delta_{\text{TDA}} = t_{\text{TDA flag}} - t_{\text{true}}, \quad \Delta_{\text{HMM}} = t_{\text{HMM flag}} - t_{\text{true}} \quad (16)$$

The TDA approach has value if $\Delta_{\text{TDA}} < \Delta_{\text{HMM}}$ systematically, with statistical significance.

4.7 Integration with downstream stages

The TDA layer outputs a regime label and confidence at each time step. VOLTA uses these to condition strategy weights: certain strategies are enabled or disabled based on regime, and edge calculations within strategies are weighted by regime conditional means.

5 Stage 4: VOLTA — Ising Portfolio Optimization

5.1 What it does

VOLTA selects an optimal subset of candidate trades from a larger universe, subject to capital, risk, and diversification constraints. The problem is naturally combinatorial: with N candidates, there are 2^N possible subsets. For realistic N in the range 50–500, this space is too large for brute-force enumeration but well-suited to Ising/QUBO formulation and simulated annealing.

5.2 Mathematical foundations

5.2.1 Binary variables and the QUBO formulation

Let $x_i \in \{0, 1\}$ for $i = 1, \dots, N$ be binary variables indicating whether candidate trade i is selected. The optimization minimizes a quadratic unconstrained binary objective:

$$H(\mathbf{x}) = - \sum_{i=1}^N e_i x_i + \sum_{k=1}^K \lambda_k \cdot P_k(\mathbf{x}) \quad (17)$$

where:

- e_i is the expected edge of trade i (in basis points of deployed capital, or similar unit)
- P_k is the k -th constraint penalty
- λ_k is the penalty weight

The minimization is over $\mathbf{x} \in \{0, 1\}^N$.

5.2.2 Standard constraint penalties

Capital constraint. Total capital deployed must not exceed C_{max} :

$$P_{\text{capital}}(\mathbf{x}) = \left(\sum_{i=1}^N c_i x_i - C_{\text{max}} \right)^2 \cdot \mathbb{I} \left[\sum_{i=1}^N c_i x_i > C_{\text{max}} \right] \quad (18)$$

where c_i is the capital required for trade i .

Vega constraint. For options portfolios, total vega exposure must stay within $\pm V_{\text{max}}$:

$$P_{\text{vega}}(\mathbf{x}) = \max \left(0, \left| \sum_{i=1}^N \nu_i x_i \right| - V_{\text{max}} \right)^2 \quad (19)$$

Sector diversification. For each sector s , limit concentration to K_s trades:

$$P_{\text{sector}}(\mathbf{x}) = \sum_{s \in S} \max \left(0, \sum_{i \in s} x_i - K_s \right)^2 \quad (20)$$

Correlation penalty. Selecting highly correlated trades increases concentration risk more than linearly. The pairwise correlation penalty is naturally quadratic:

$$P_{\text{corr}}(\mathbf{x}) = \sum_{i < j} \tilde{C}_{ij}^2 \cdot x_i x_j \quad (21)$$

where $\tilde{\mathbf{C}}$ is the cleaned correlation matrix from Stage 2.

5.2.3 Regime-conditional edges

The expected edges e_i are conditioned on the current regime r from Stage 3:

$$e_i(r) = \alpha_i \cdot \mu_i(r) + (1 - \alpha_i) \cdot \mu_i^{(\text{base})} \quad (22)$$

where $\mu_i(r)$ is the historical mean edge of strategy class containing trade i during regime r , $\mu_i^{(\text{base})}$ is the unconditional mean, and $\alpha_i \in [0, 1]$ is a regime-confidence weight. If the regime classifier is highly confident, α_i is close to 1.

5.2.4 Strategy-type diversity

To prevent concentration in correlated strategy types (e.g., all premium-selling positions), enforce diversity:

$$P_{\text{type}}(\mathbf{x}) = \sum_{\tau \in T} \max \left(0, \sum_{i: \text{type}(i)=\tau} x_i - K_\tau \right)^2 \quad (23)$$

5.3 Full objective

Combining all terms:

$$H(\mathbf{x}) = - \sum_i e_i(r) x_i + \lambda_C P_{\text{capital}}(\mathbf{x}) + \lambda_V P_{\text{vega}}(\mathbf{x}) + \lambda_S P_{\text{sector}}(\mathbf{x}) + \lambda_T P_{\text{type}}(\mathbf{x}) + \lambda_\rho P_{\text{corr}}(\mathbf{x}) \quad (24)$$

The λ weights are hyperparameters tuned through backtesting. Too small: constraints get violated. Too large: optimizer minimizes penalty rather than maximizing edge.

5.4 Equivalence to Ising model

The QUBO formulation is equivalent to an Ising model under the transformation $s_i = 2x_i - 1$, mapping binary variables in $\{0, 1\}$ to spins in $\{-1, +1\}$. The Ising Hamiltonian:

$$H_{\text{Ising}}(\mathbf{s}) = - \sum_i h_i s_i - \sum_{i < j} J_{ij} s_i s_j \quad (25)$$

The coefficients h_i (local fields) and J_{ij} (couplings) derive from the QUBO coefficients via the transformation. This equivalence is why specialized Ising hardware (D-Wave annealers, coherent Ising machines from Quanfluent) can solve VOLTA's optimization problems.

5.5 Solver: simulated annealing

For the problem sizes typical in VOLTA (50 to 5000 variables), simulated annealing on classical hardware is sufficient. The algorithm samples spin configurations probabilistically, with acceptance probability for energy-increasing moves decreasing as a temperature parameter cools.

Algorithm 4 Simulated annealing for QUBO solving

Require: QUBO objective H , initial temperature T_0 , cooling rate γ , iterations M

Ensure: Approximate ground state $\mathbf{x}^*$

```

1: Initialize  $\mathbf{x}$  uniformly at random in  $\{0, 1\}^N$ 
2:  $T \leftarrow T_0$ 
3:  $\mathbf{x}^* \leftarrow \mathbf{x}$ 
4: for  $m = 1$  to  $M$  do
5:   Pick random index  $i$ 
6:   Flip:  $\mathbf{x}' \leftarrow \mathbf{x}$  with  $x'_i = 1 - x_i$ 
7:    $\Delta H \leftarrow H(\mathbf{x}') - H(\mathbf{x})$ 
8:   if  $\Delta H \leq 0$  or  $\text{rand}() < \exp(-\Delta H/T)$  then
9:      $\mathbf{x} \leftarrow \mathbf{x}'$ 
10:  end if
11:  if  $H(\mathbf{x}) < H(\mathbf{x}^*)$  then
12:     $\mathbf{x}^* \leftarrow \mathbf{x}$ 
13:  end if
14:   $T \leftarrow T \cdot \gamma$ 
15: end for
16: return  $\mathbf{x}^*$ 

```

For high-quality solutions, run multiple annealing chains with different random seeds, returning the best across all chains.

5.6 Implementation

The D-Wave `dimod` library provides QUBO data structures, and `neal` provides a fast simulated annealing solver. Listing 3 shows the core formulation.

```

1 import dimod
2 from neal import SimulatedAnnealingSampler
3
4 def build_qubo(candidates, constraints, regime):
5     """
6     Build a QUBO for VOLTA's portfolio selection problem.
7
8     Args:
9         candidates: list of candidate trades with edge, capital, vega, sector
10        constraints: dict with capital_max, vega_max, sector_caps, etc.
11        regime: regime label from Stage 3 for edge conditioning
12
13    Returns:
14        dimod.BinaryQuadraticModel
15    """
16    bq = dimod.BinaryQuadraticModel("BINARY")
17
18    # Linear terms (edge maximization)
19    for i, c in enumerate(candidates):
20        edge = c.expected_edge_for_regime(regime)
21        bq.add_variable(i, -edge)
22
23    # Capital constraint (quadratic when expanded)

```



```

24 lam_capital = 100.0 # tune via backtesting
25 C_max = constraints["capital_max"]
26 for i, ci in enumerate(candidates):
27     bqm.add_variable(i, lam_capital * ci.capital**2)
28     for j in range(i+1, len(candidates)):
29         cj = candidates[j]
30         bqm.add_interaction(i, j, 2 * lam_capital * ci.capital * cj.
31                             capital)
32
33 # Correlation penalty (uses cleaned correlation from Stage 2)
34 lam_corr = 10.0
35 for i in range(len(candidates)):
36     for j in range(i+1, len(candidates)):
37         rho = candidates[i].correlation_with(candidates[j])
38         bqm.add_interaction(i, j, lam_corr * rho**2)
39
40 # Sector caps, vega caps, type diversity ... (similarly added)
41
42 return bqm
43
44 def solve_volta(candidates, constraints, regime, num_reads=1000):
45     bqm = build_qubo(candidates, constraints, regime)
46     sampler = SimulatedAnnealingSampler()
47     sampleset = sampler.sample(bqm, num_reads=num_reads)
48     best = sampleset.first.sample
49     return [i for i in range(len(candidates)) if best[i] == 1]

```

Listing 3: VOLTA QUBO formulation

5.7 Solver comparison

Figure 5 compares classical simulated annealing against greedy heuristics on a synthetic VOLTA instance.

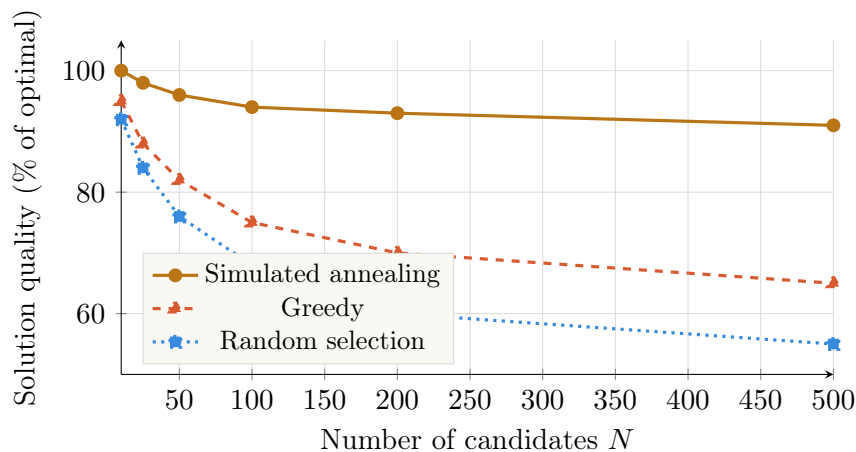


Figure 5: Solution quality versus problem size for three approaches. Simulated annealing maintains near-optimal solutions even at $N = 500$ where greedy approaches lose 30%+ of available edge.

5.8 Integration with downstream stages

VOLTA's output is a trade plan: which candidates to take, at what sizes. The plan flows to execution (manual or automated) and to the AI agent layer for narrative synthesis.

6 Stage 5: AI Agent Meta Layer

6.1 What it does

The AI agent layer is cross-cutting rather than sequential. It observes outputs from every stage and performs three distinct functions:

Validation. After each stage produces output, an agent reviews for sanity. If TDA suddenly flags a regime change with high confidence, an agent checks news and pattern memory for supporting evidence. If VOLTA selects unusual trades, an agent reviews the reasoning.

Hypothesis generation. The pattern memory engine surfaces correlations. Agents formulate testable hypotheses from those correlations, propose simple statistical tests, and flag promising results for human review.

Narrative synthesis. The system produces large amounts of numerical output. Agents translate this into human-readable summaries explaining what the system did and why.

6.2 Architecture

Figure 6 shows how agents interact with the rest of the pipeline.

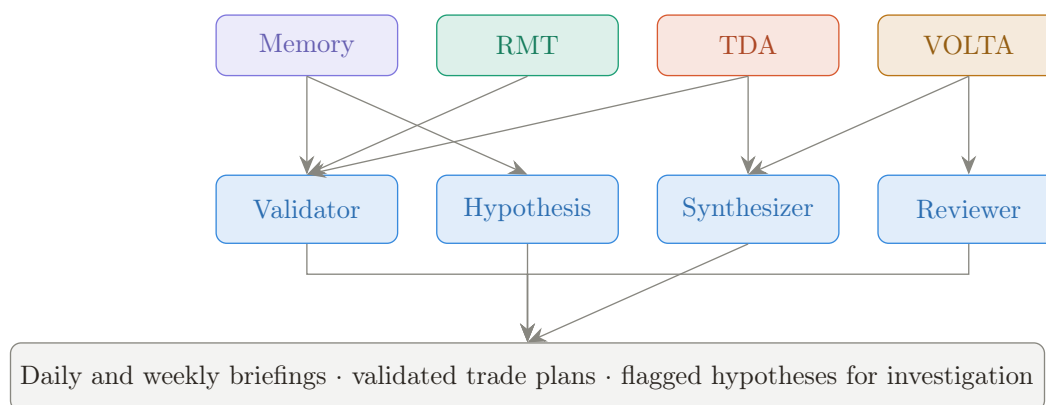


Figure 6: The AI agent layer observes outputs from all upstream stages, performs validation, hypothesis generation, and synthesis, then produces human-readable briefings.

6.3 Validation logic

The validator agent checks output consistency between stages. Pseudo-code is shown in Algorithm 5.

6.4 Implementation

Agent calls use the Anthropic Claude API directly. Each agent has a specific role prompt and receives only the data relevant to its task. Coordination between agents is handled by simple Python orchestration — avoid heavyweight frameworks like LangChain for projects of this scope.

7 Cross-Cutting Concerns

7.1 Data architecture

Three storage layers serve different purposes (Table 1).

Algorithm 5 Cross-stage output validation**Require:** Outputs from all stages at time t : $\{M_t, R_t, T_t, V_t\}$ **Ensure:** Validation report with anomaly flags

```

1: anomalies  $\leftarrow \emptyset$ 
2: if TDA regime change flagged at high confidence then
3:   Query memory: did news, filings, or pattern triggers occur?
4:   if no supporting evidence then
5:     Add anomaly: Unexplained regime change
6:   end if
7: end if
8: if VOLTA selects trade unusual for current regime then
9:   Query: what override caused selection?
10:  Add anomaly: Regime-strategy mismatch if no clear override
11: end if
12: if VOLTA's expected edge differs from historical mean by  $> 2\sigma$  then
13:  Add anomaly: Edge calibration drift
14: end if
15: return anomalies

```

Layer	Technology	Purpose
Time-series	PostgreSQL TimescaleDB	+ Raw price data, options chains, returns, indicators. Billions of rows.
Graph	Neo4j Community	Pattern memory: entities, relationships, events.
Computational artifacts	Parquet files on disk	Cleaned matrices, persistence diagrams, feature vectors. Immutable, query-sequentially.

Table 1: Three-tier storage architecture. Each layer's design matches its query patterns.

7.2 Repository structure

The codebase follows a clear stage-based structure to keep components decoupled:

```

1 quantflow/
2   data/                # data ingestion adapters
3     adapters/          # one file per data source
4     schemas/           # common internal format
5     pipeline.py        # orchestrates data flow
6   memory/              # Stage 1: pattern memory
7     extraction/        # LLM-based extractors
8     storage/           # Neo4j and PostgreSQL clients
9     detection/         # pattern detection jobs
10  rmt/                  # Stage 2: RMT correlation cleaning
11    filter.py           # Marchenko-Pastur implementation
12    rolling.py          # rolling window computation
13  tda/                  # Stage 3: TDA regime detection
14    features.py         # persistent homology features
15    classifier.py       # regime classification model
16    evaluation.py       # latency and accuracy metrics
17  volta/                # Stage 4: Ising optimizer
18    formulation.py      # QUBO problem builder
19    solver.py           # simulated annealing wrapper
20    candidates.py       # signal engine integration
21  agents/               # Stage 5: AI agent layer
22    validators.py       # output validation
23    synthesizers.py     # narrative generation

```

```

24 hypothesis.py      # hypothesis generation
25 backtest/          # backtesting framework
26 notebooks/         # research notebooks
27 tests/             # test suite
28 scripts/           # one-off scripts
29 pyproject.toml      # dependencies
30 README.md

```

7.3 Reproducibility

Every output must be reproducible from its inputs. This requires:

- Deterministic random seeds throughout
- Git commit hashes recorded in every output file
- Frozen dependency versions in `pyproject.toml`
- Data snapshots captured at the time of each computation
- Configuration as code (no hand-tuned magic numbers in scripts)

7.4 Testing strategy

Each stage requires both unit tests (does the math work?) and integration tests (does the stage work with realistic inputs?). Budget 30–40% of development time for testing.

For Stage 2 (RMT), validate against published examples from Bouchaud and Potters. For Stage 3 (TDA), reproduce results from a published TDA-on-markets paper before claiming novelty. For Stage 4 (VOLTA), compare optimizer output against brute force on small problems where ground truth is known.

8 Build Sequence

The full pipeline takes approximately 12–16 weeks of focused work to implement at production quality. The sequence in Table 2 prioritizes building the components in an order where each subsequent stage has clean inputs from previous stages.

9 Validation Methodology

9.1 Stage-by-stage validation

Each stage has specific validation criteria:

- **Stage 1 (Memory):** Manual audit of 100 random extractions against source documents. Target: precision $> 95\%$, recall $> 80\%$.
- **Stage 2 (RMT):** Compare Frobenius distance between cleaned and ground-truth covariances on synthetic data. Target: cleaning improves accuracy by $> 30\%$.
- **Stage 3 (TDA):** Detection latency on historical regimes. Target: TDA flag precedes HMM flag by 3+ days on majority of regime changes.
- **Stage 4 (VOLTA):** Backtested Sharpe ratio versus greedy baseline. Target: $\geq 15\%$ Sharpe improvement.
- **Stage 5 (Agents):** Anomaly flagging precision on adversarial test cases. Target: $> 80\%$ precision, false positive rate $< 5\%$.

Phase	Components	Deliverable
Weeks 1–2	Data pipeline, schemas, adapters	Clean data flowing through to storage
Weeks 3–4	Stage 2 RMT implementation, validation against published examples	Cleaned correlation matrices available as time series
Weeks 5–6	Stage 1 pattern memory: graph schema, LLM extraction	Pattern memory accepting daily documents, extracting structured observations
Weeks 7–9	Stage 3 TDA: feature extraction, classifier training	Working regime detector, validated on historical regimes
Weeks 10–12	Stage 4 VOLTA: QUBO formulation, solver, regime integration	End-to-end trade plan generation
Weeks 13–14	Stage 5 agents: validation, synthesis, briefings	Daily briefing pipeline running
Weeks 15–16	Integration testing, backtesting, documentation	Reproducible end-to-end results

Table 2: 16-week build sequence. Each phase produces a concrete deliverable that downstream phases can consume.

9.2 End-to-end validation

The full pipeline must be validated on out-of-sample data. Reserve the most recent 12 months as a hold-out set, never touched during development. Measure realized P&L of the system’s recommended trades versus a benchmark (equal-weighted candidate set or buy-and-hold of the universe). Statistical significance requires a track record of at least 12 months of paper-traded or live data.

Summary

This document specifies a five-stage quantitative research pipeline:

1. **Pattern memory** accumulates observations from heterogeneous sources into a graph database. Compounds in value over time.
2. **RMT cleaning** removes statistical noise from sample correlation matrices using Marchenko-Pastur eigenvalue filtering. Quick to implement, immediately valuable.
3. **TDA regime detection** computes persistent homology features on cleaned correlation matrices, classifies regimes, detects transitions earlier than classical methods. The novel research contribution.
4. **VOLTA Ising optimization** selects trades from candidates using QUBO formulation with regime-conditional edges and constraint penalties. Production-quality decisions.
5. **AI agents** validate outputs, generate hypotheses, synthesize narratives. Cross-cutting reasoning layer.

The system is designed to be market-agnostic and runs primarily in Python. Total build time at focused effort is approximately 16 weeks. Each stage is independently valuable and can be deployed incrementally.

End of document.